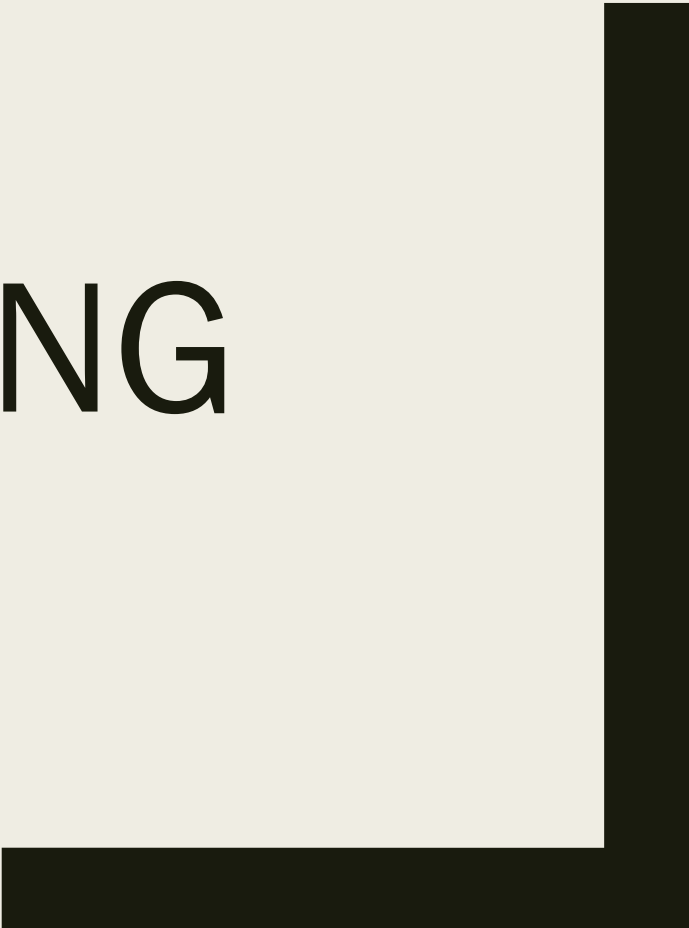




DEEP LEARNING



Outline

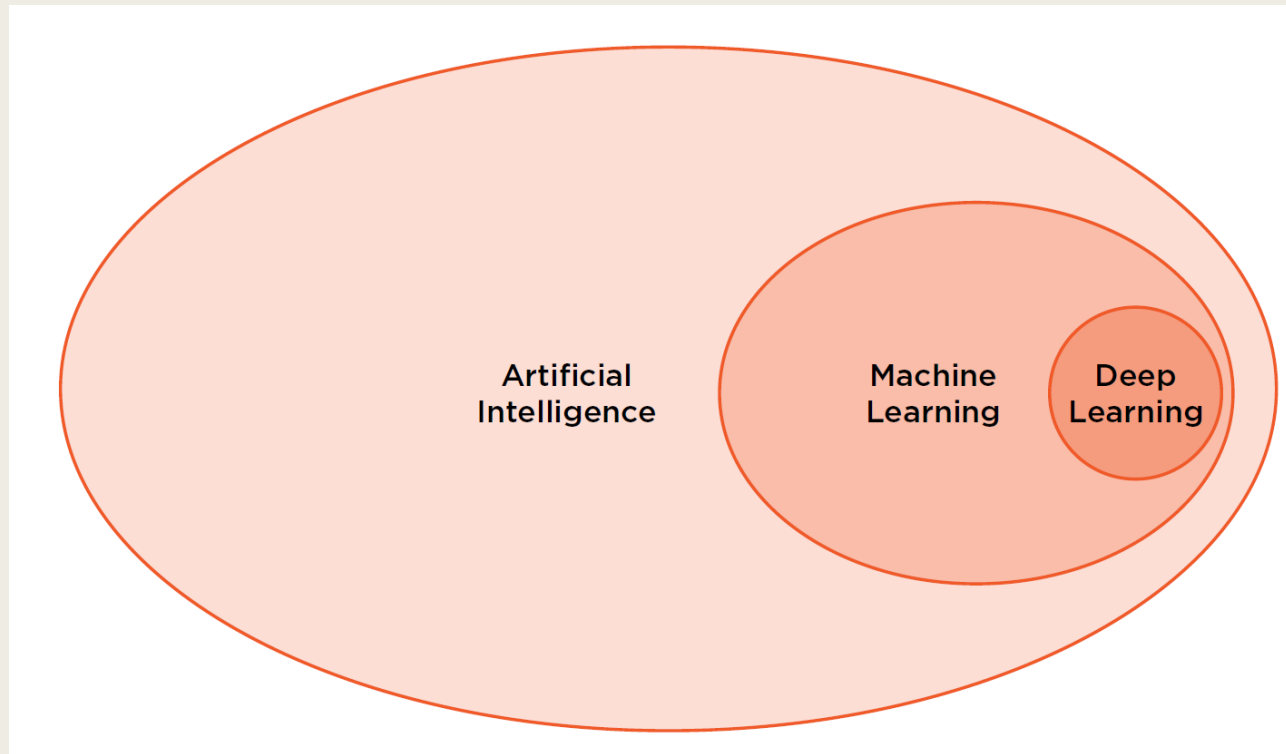
- What is Deep Learning?
- An Artificial Neural Network
- Components of Neural Networks
- Types: Fully connected Neural Networks, CNNs, Transfer Learning
- Data Processing, Metrics, and Tuning
- Challenges in Deep Learning
- Deep Learning Frameworks

Deep Learning

- Artificial Neural networks, conceived in the '50s as a crude approximation of how the human brain works, are the basis of what is now referred to as Deep Learning.
- Artificial Neural Networks used to address machine learning problems are referred to as Deep Learning. Since our goal is to apply neural networks to solve machine learning problems, we will use the terms Deep Learning and Neural Networks interchangeably.
- Deep Learning is a form of *Artificial Intelligence* that uses a type of machine learning called an *artificial neural network* with *multiple hidden layers* that learns *hierarchical representations* of the underlying data in order to make *predictions* given new data

Deep Learning

is a Form of Artificial Intelligence



Evaluation of Deep Learning

Pros

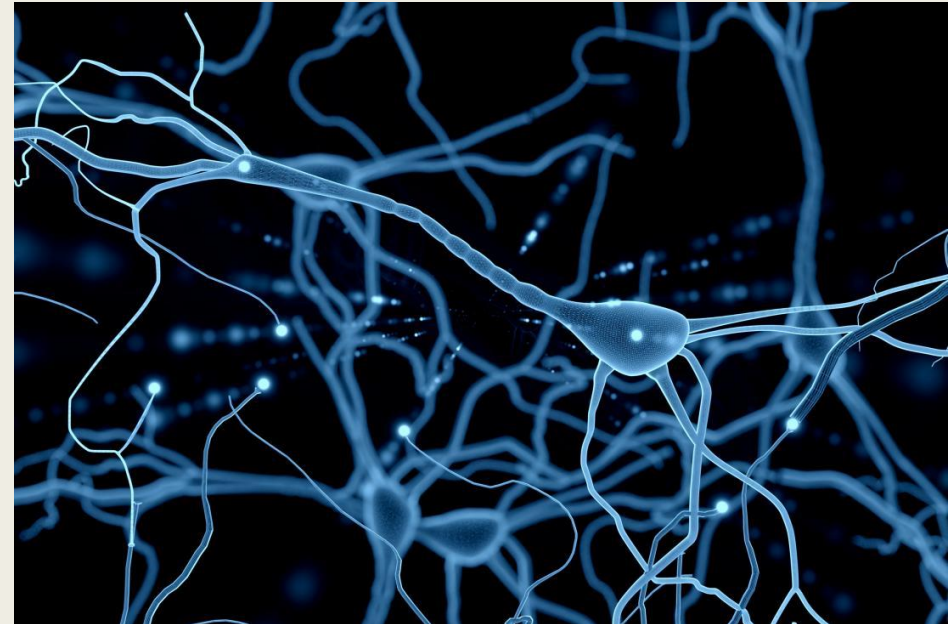
- Excels at tasks such as computer vision, natural language processing, speech recognition.
- Powers many recommender systems, and fraud detection systems.
- Algorithms are very general and adaptive
- Work directly on raw data. No feature engineering required.

Cons

- Computational resource intensive
- Tricky hyper parameterization
- Non-optimal methods

Artificial Neural Network

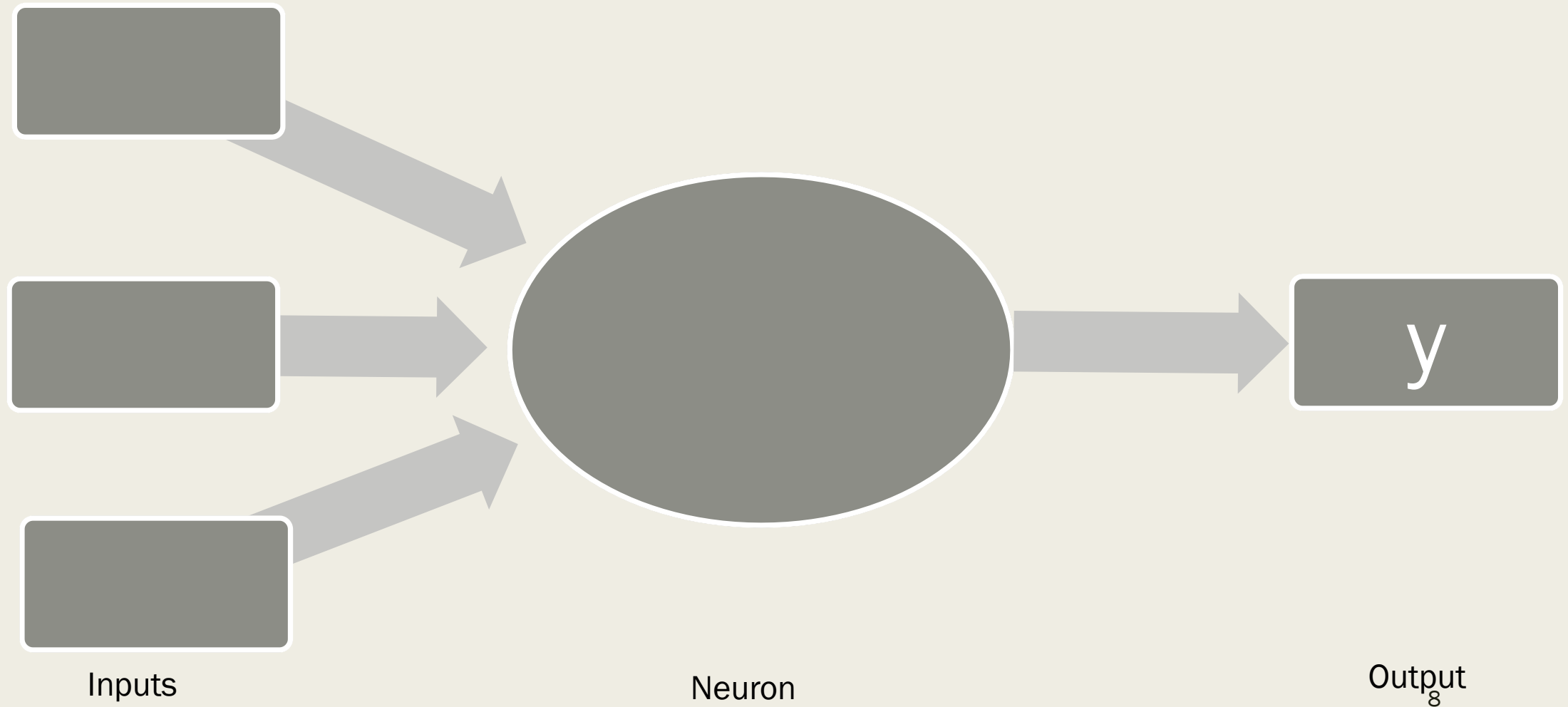
- Inspired by the biological neuron but they work very differently
- Consists of a series of Neurons connected together in a network
- Lets first examine an Artificial Neuron.



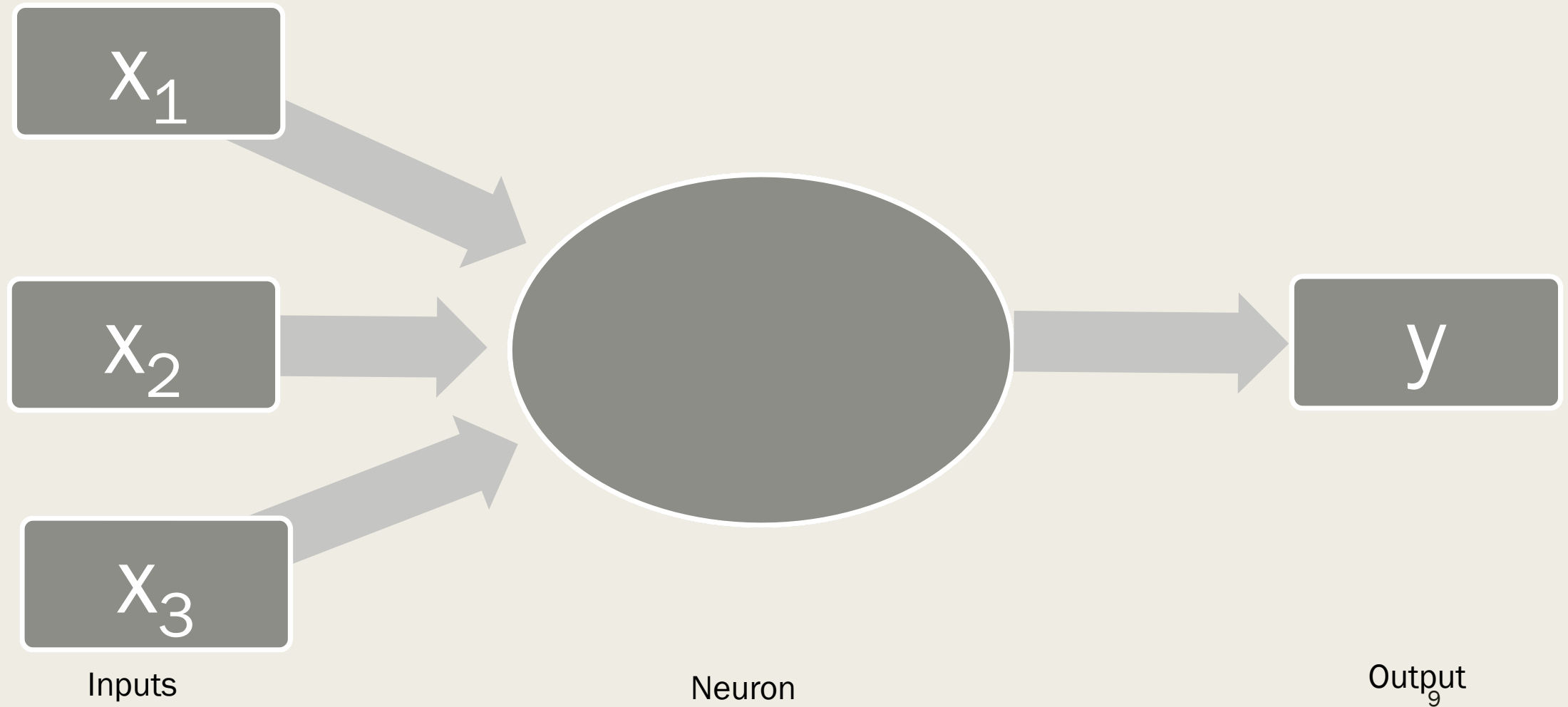
Artificial Neuron

- Input (x_1, x_2, x_3)
- Weights or parameters ($\omega_1, \omega_2, \omega_3$)
- Bias (ω_0)
- Neuron
- Output (y)
- Activation function (e.g., $\tanh$)

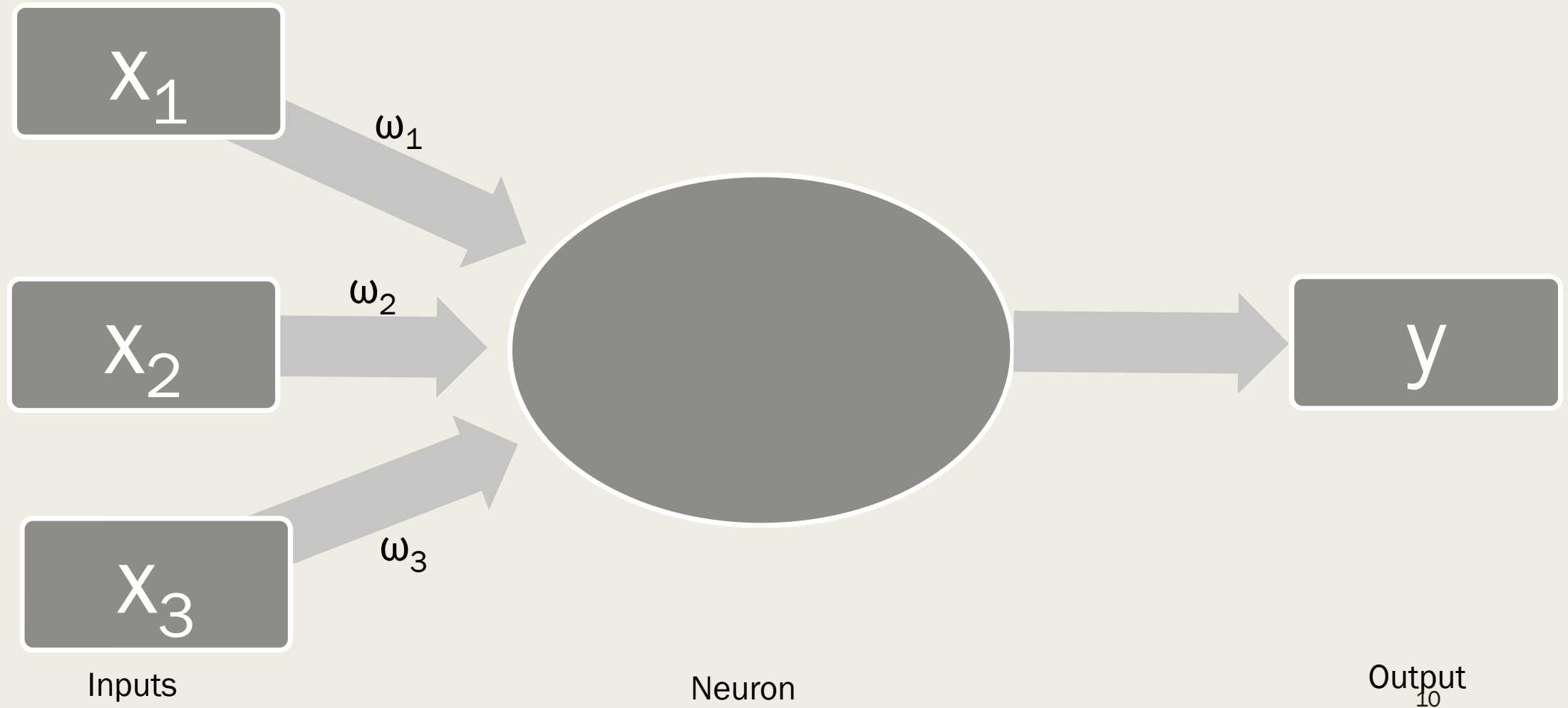
Artificial Neuron



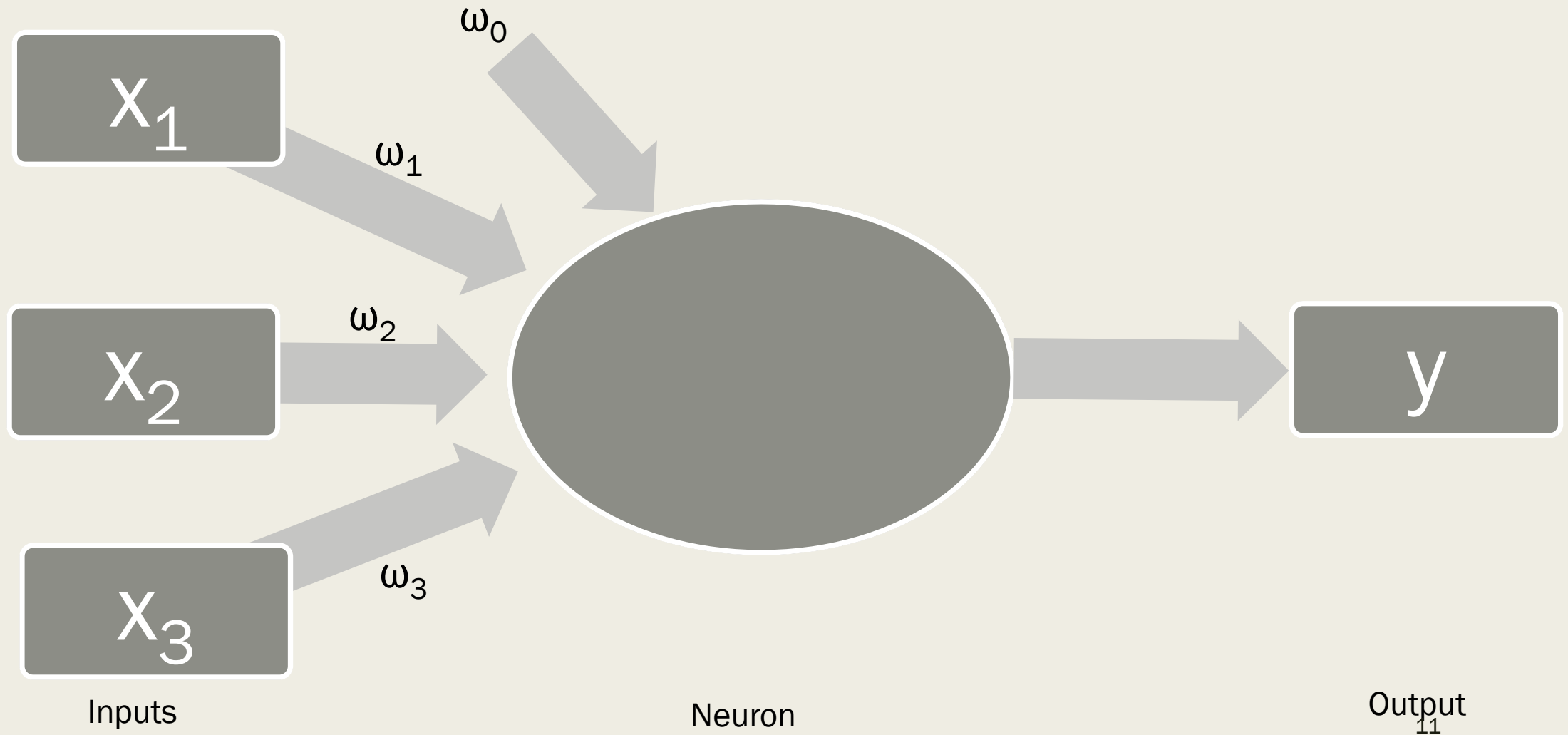
Artificial Neuron



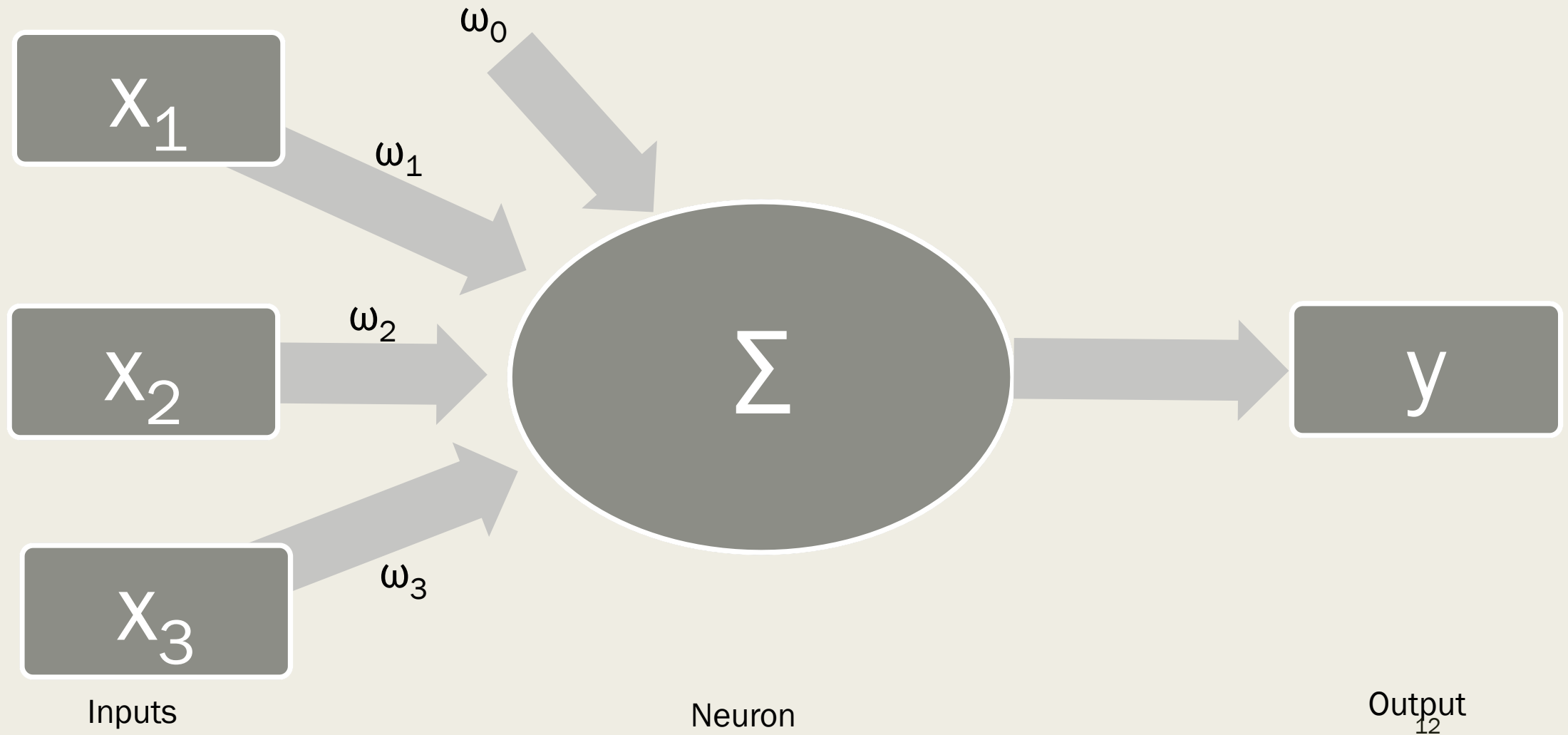
Artificial Neuron



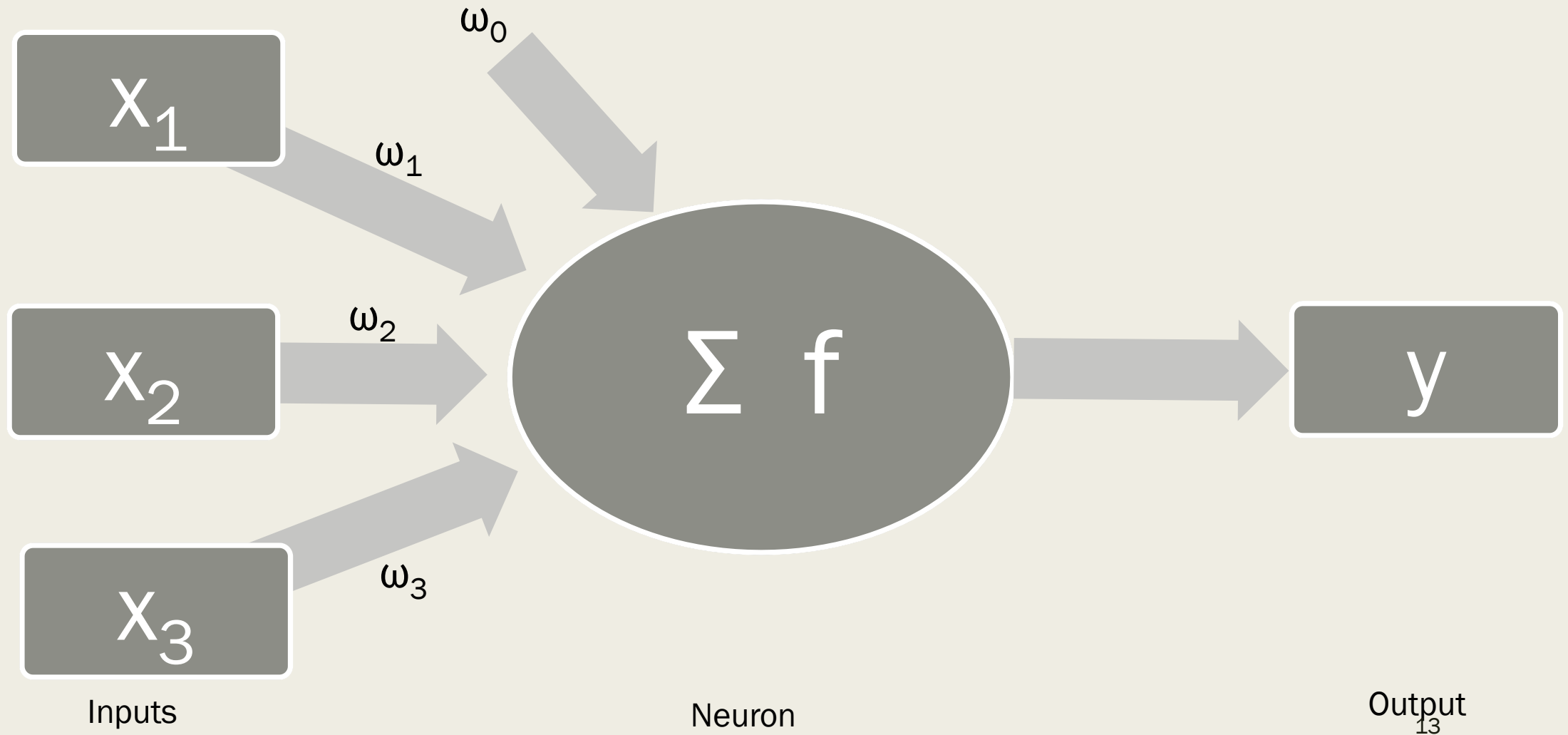
Artificial Neuron



Artificial Neuron



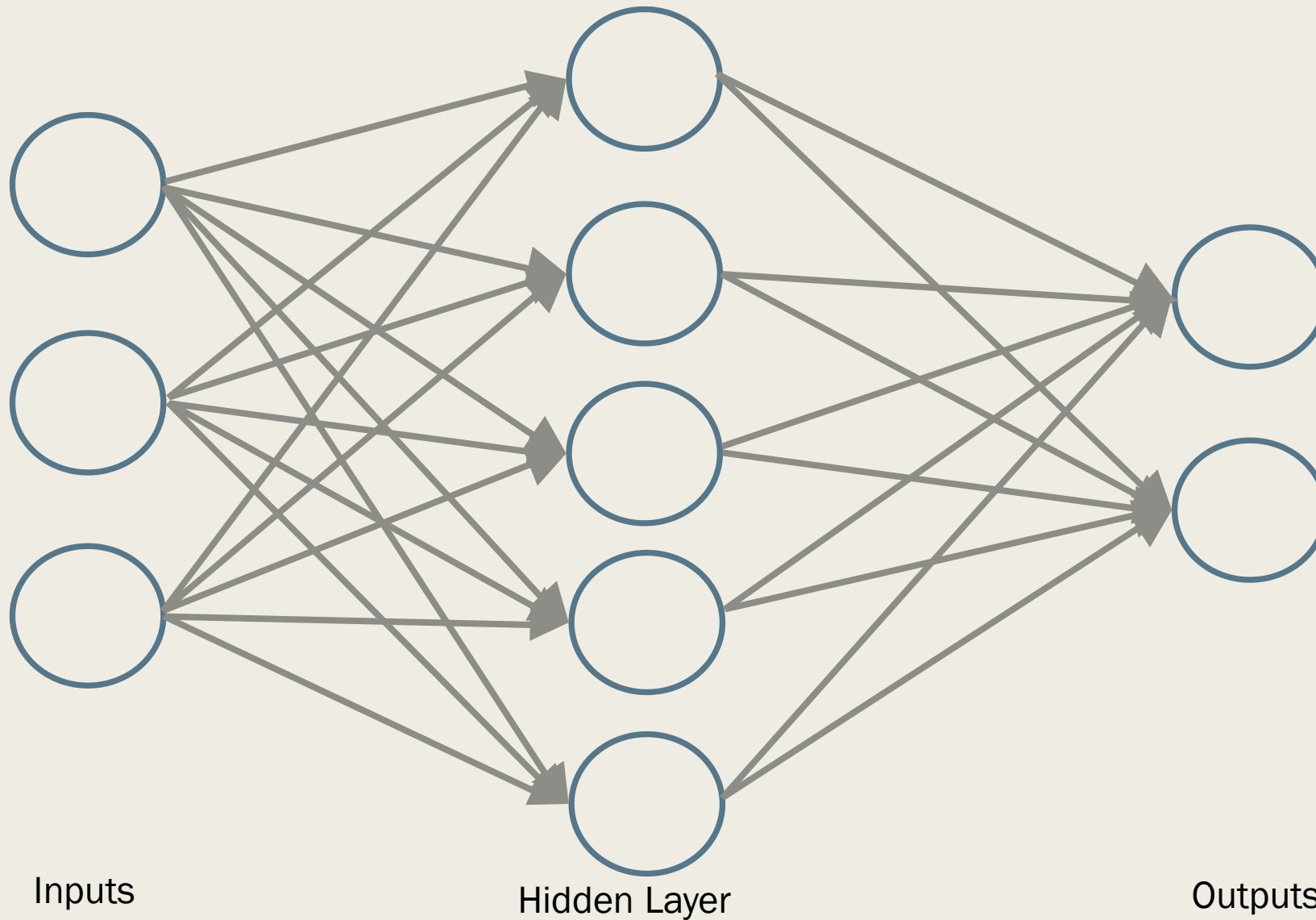
Artificial Neuron



Artificial Neural Network

- Network of connected neurons
- Includes
 - *Input Layer*
 - *One or more Hidden Layer(s)*
 - *Output Layer*

Artificial Neural Network



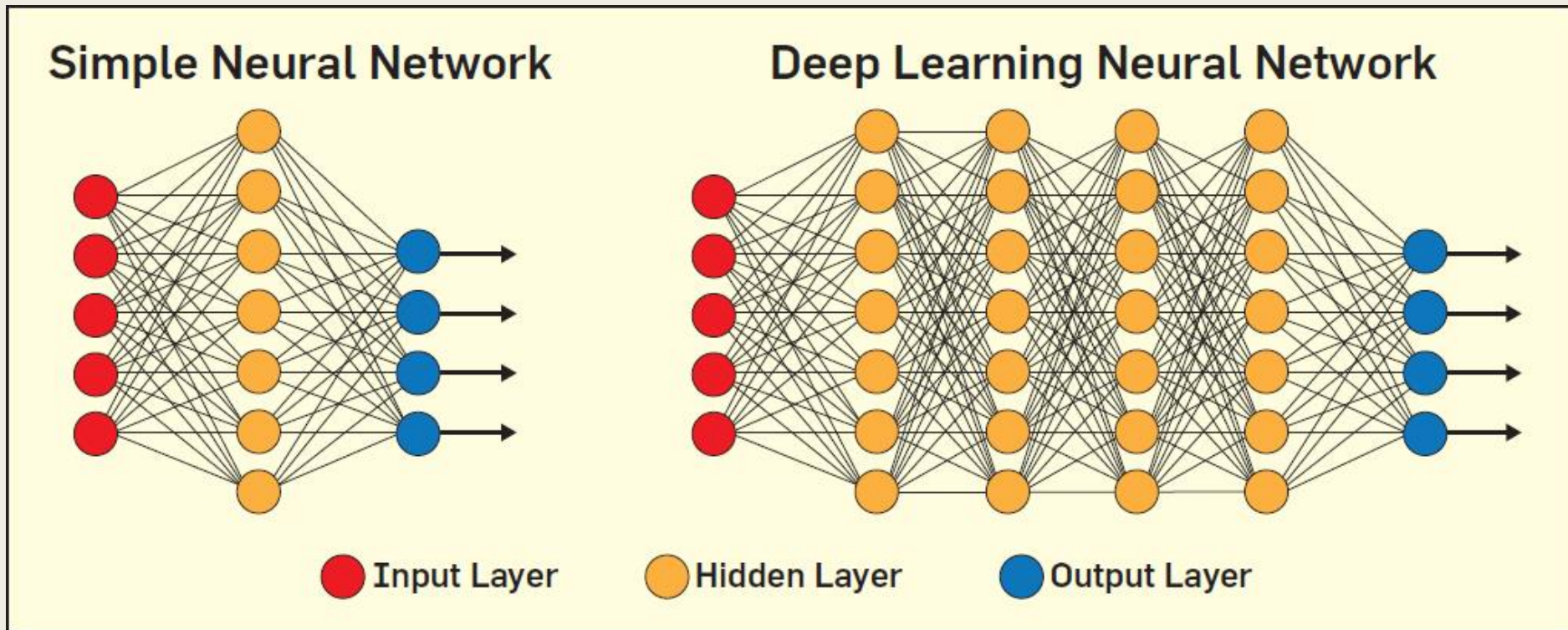
Artificial Neural Network

- In practice, each layer can be a
 - *vector (one-dimensional)*
 - *matrix (two-dimensional array)*
 - *tensor (n-dimensional array)*

Deep Learning Neural Network

- A neural network with more than one hidden layer
- Adding more hidden layers enables the network to model progressively more complex functions

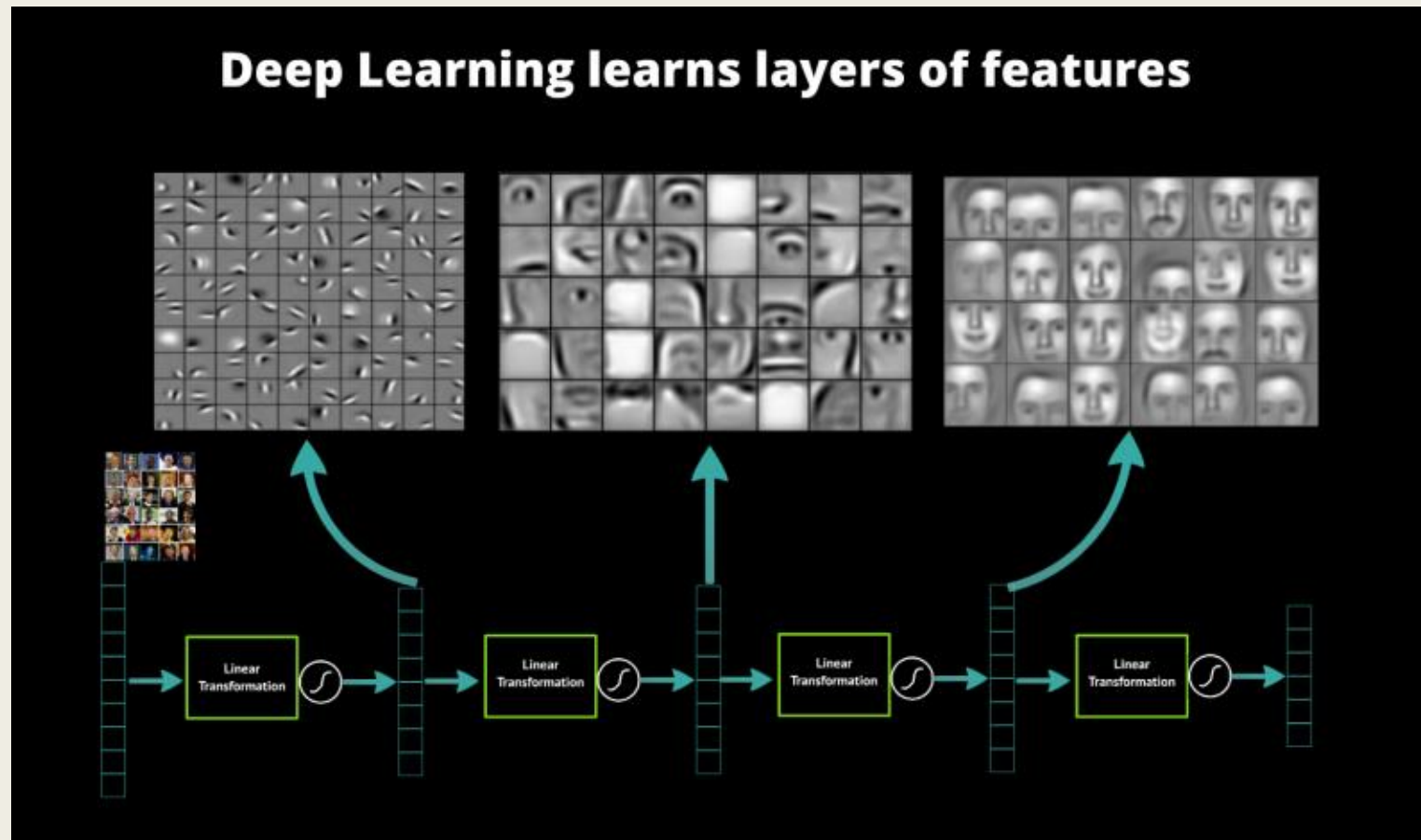
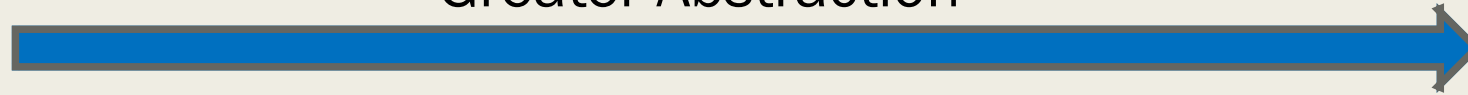
Deep Learning Neural Network



Source: [Communications of the ACM](#)

Hierarchical Representations

Greater Abstraction



COMPONENTS

Neural Network Architecture

- Input Layer
 - *Raw data representation*
 - *No computation, just data distribution*
- Hidden Layers
 - *Feature extraction and transformation*
 - *Multiple layers enable hierarchical learning*
- Output Layer
 - *Final predictions or classifications*
 - *Structure depends on task type*

Universal Approximation Theorem

- A feedforward network with a single hidden layer can approximate any continuous function
- Thus, neural networks are theoretically capable of learning any mapping
- But it may require an exponentially large number of neurons

Activation Functions

- Non-linearity: Enables learning of complex patterns
- Gradient Flow: Affects training dynamics
- Output Range: Determines neuron behavior

Common Activation Functions

- ReLU (Rectified Linear Unit)
 - $f(x) = \max(0, x)$
- Sigmoid
 - $f(x) = 1/(1 + e^{(-x)})$
- Tanh (Hyperbolic Tangent)
 - $f(x) = (e^x - e^{(-x)})/(e^x + e^{(-x)})$

Training Neural Networks: Learning Process

- Supervised Learning Framework
 - *Input: Training data (X, y)*
 - *Goal: Learn mapping $f: X \rightarrow y$*
 - *Process: Minimize loss function $L(f(X), y)$*
- Gradient Descent Algorithm
 1. *Forward Pass: Compute predictions*
 2. *Loss Calculation: Measure prediction error*
 3. *Backward Pass: Compute gradients*
 4. *Parameter Update: Adjust weights using gradients*
- Mathematical Foundation
 - *Loss Function: Measures prediction error (e.g., MSE, Cross-entropy)*
 - *Gradient: Direction of steepest increase in loss*
 - *Learning Rate: Step size for parameter updates*

Backpropagation Algorithm

- To efficiently compute gradients for all parameters
- Relies on chain rule of calculus
- Algorithm Steps
 1. *Forward Pass: Compute activations for all layers*
 2. *Backward Pass:*
 - Compute output layer gradients
 - Propagate gradients backward through layers
 - Apply chain rule at each layer
- Chain Rule Application: $\partial L / \partial w = \partial L / \partial a \times \partial a / \partial z \times \partial z / \partial w$
 - where: L = loss, a = activation, z = weighted sum, w = weight

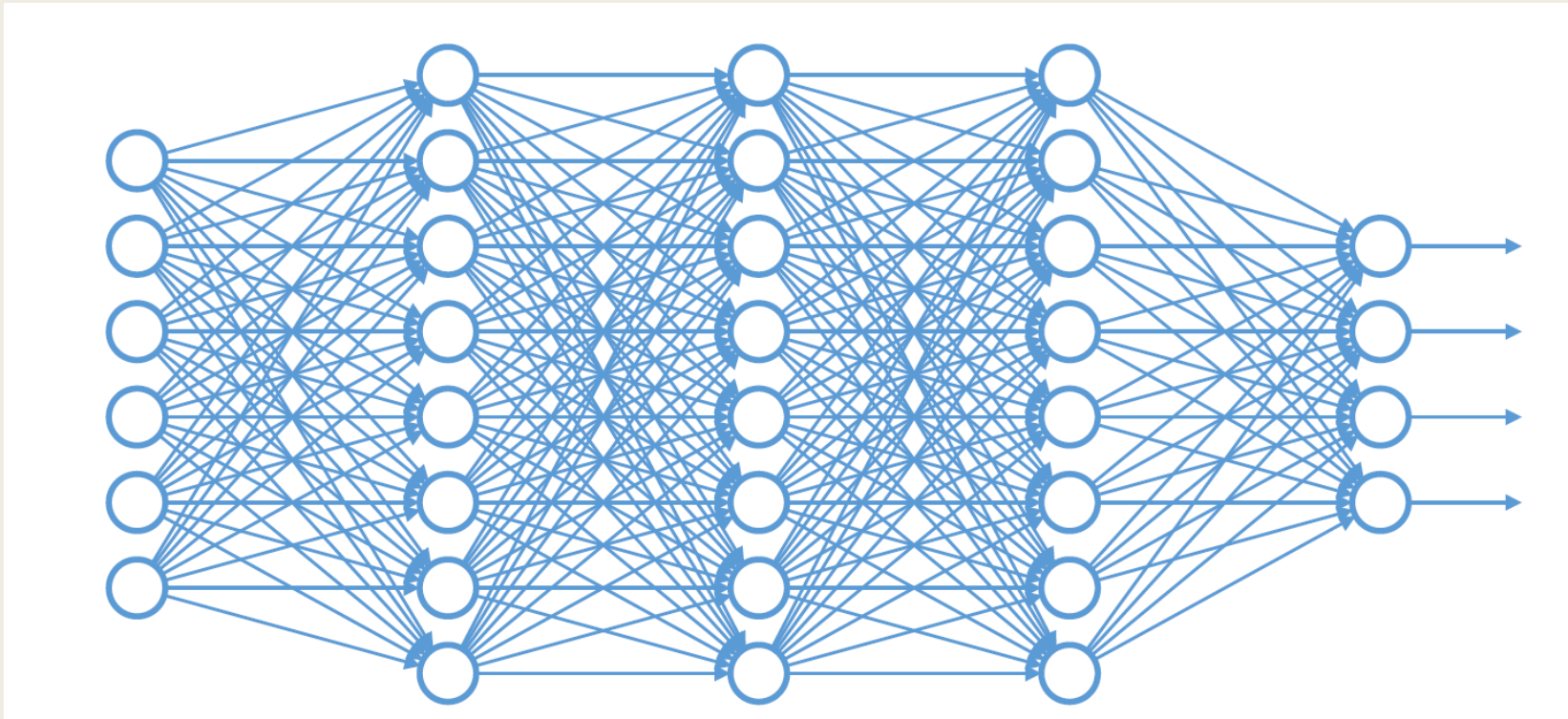
Optimization Algorithms

- Variations of Gradient Descent
- Stochastic Gradient Descent (SGD)
 - *Updates parameters after each training example*
 - *Advantages: Fast convergence, escapes local minima*
 - *Disadvantages: High variance, noisy updates*
- Mini-batch Gradient Descent
 - *Updates after processing a batch of examples*
 - *Advantages: Balance between speed and stability*
 - *Disadvantages: Requires tuning batch size*
- Adam Optimizer
 - *Adaptive learning rates for each parameter*
 - *Advantages: Fast convergence, handles sparse gradients*
 - *Disadvantages: More hyperparameters to tune*

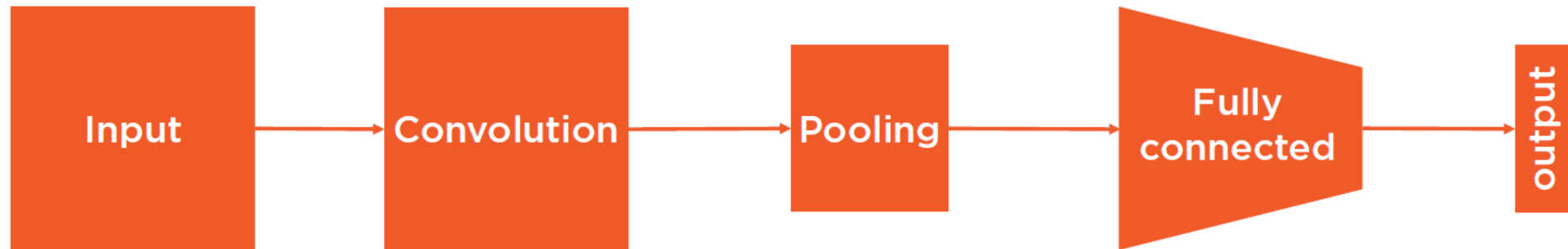
Regularization Techniques

- Designed to address overfitting problem often related to model complexity.
- Common regularization methods
- Dropout
 - *Mechanism: Randomly set neurons to zero during training*
 - *Effect: Prevents co-adaptation, improves generalization*
 - *Rate: Typically 0.2-0.5*
- Weight Decay (L2 Regularization)
 - *Mechanism: Add penalty term to loss function*
 - *Effect: Prevents large weight values*
 - *Formula: $L' = L + \lambda ||w||^2$*
- Early Stopping
 - *Mechanism: Stop training when validation performance degrades*
 - *Effect: Prevents overfitting to training data*

Fully Connected Feed-Forward Network



Convolutional Neural Network



Convolutional Neural Networks (CNNs)

- Fully connected networks don't scale well for images. CNNs address this by using local connectivity and parameter sharing
- Key Components:
 - *Convolutional Layers*
 - Detect local patterns (edges, textures, shapes)
 - *Pooling Layers*
 - Reduce spatial dimensions, increase robustness. Key types are max pooling, average pooling

CNN Architecture

■ Architectural principles

- *Progressive Complexity: Simple features → complex features*
- *Spatial Reduction: Pooling layers reduce dimensions*
- *Feature Hierarchy: Low-level → high-level features*

■ Common Architectures

- *LeNet-5 (1998): Used for Handwritten digit recognition. First successful CNN for image classification*
- *AlexNet (2012): ImageNet classification. Deep CNN with ReLU activation, dropout*
- *ResNet (2015): Image classification. Residual connections, very deep networks*

Transfer Learning

- Using knowledge learned from one task to improve performance on a related task
- Reduces training time, improves performance, and requires less data

Transfer Learning Strategies

■ Feature Extraction

- *Use pre-trained features, train only new classifier*
- *Used when data is limited data and domains are similar*
- *Benefit is fast training and good performance*

■ Fine-tuning

- *Update some pre-trained layers*
- *Use when more is data available and domain adaptation is needed*
- *Benefit is better adaptation to target domain*

Data Processing and Augmentation

- Data Preprocessing
 - *Normalize features (to similar ranges) or standardize.*
 - *Helps improve training stability and convergence*
- Data Augmentation
 - *Creating variations of training data*
 - *Increases effective dataset size and improves generalization*
- Common Augmentation Techniques
 - *Geometric: Rotation, translation, scaling, flipping*
 - *Color: Brightness, contrast, saturation adjustment*
 - *Noise: Gaussian noise, salt-and-pepper noise*
- Benefits
 - *Regularization: Prevents overfitting*
 - *Robustness: Improves model generalization*
 - *Efficiency: Reduces need for more training data*

Model Evaluation and Metrics

- Training vs. Validation vs. Test Sets
 - *Training Set: Used for learning model parameters*
 - *Validation Set: Used for hyperparameter tuning and early stopping*
 - *Test Set: Used for final performance assessment*
- Common Metrics
- Regression Metrics
 - *Mean Squared Error (MSE): Average squared prediction error*
 - *Mean Absolute Error (MAE): Average absolute prediction error*
 - *R^2 Score: Proportion of variance explained*
- Classification Metrics
 - *Accuracy: Proportion of correct predictions*
 - *Precision: $\text{True positives} / (\text{True positives} + \text{False positives})$*
 - *Recall: $\text{True positives} / (\text{True positives} + \text{False negatives})$*
 - *F1-Score: Harmonic mean of precision and recall*

Hyperparameter Tuning

- Hyperparameters are parameters set before training. Examples include learning rate, batch size, number of layers, dropout rate
- Tuning Strategies
 - *Grid Search*
 - Try all combinations of hyperparameter values.
 - Systematic, guaranteed to find best in grid but computationally expensive
 - *Random Search*
 - Randomly sample hyperparameter combinations
 - More efficient than grid search but may miss optimal values
 - *Bayesian Optimization*
 - Use probabilistic model to guide search
 - Most efficient for expensive evaluations but implementation is more complex

Challenges in Deep Learning

■ Technical Challenges

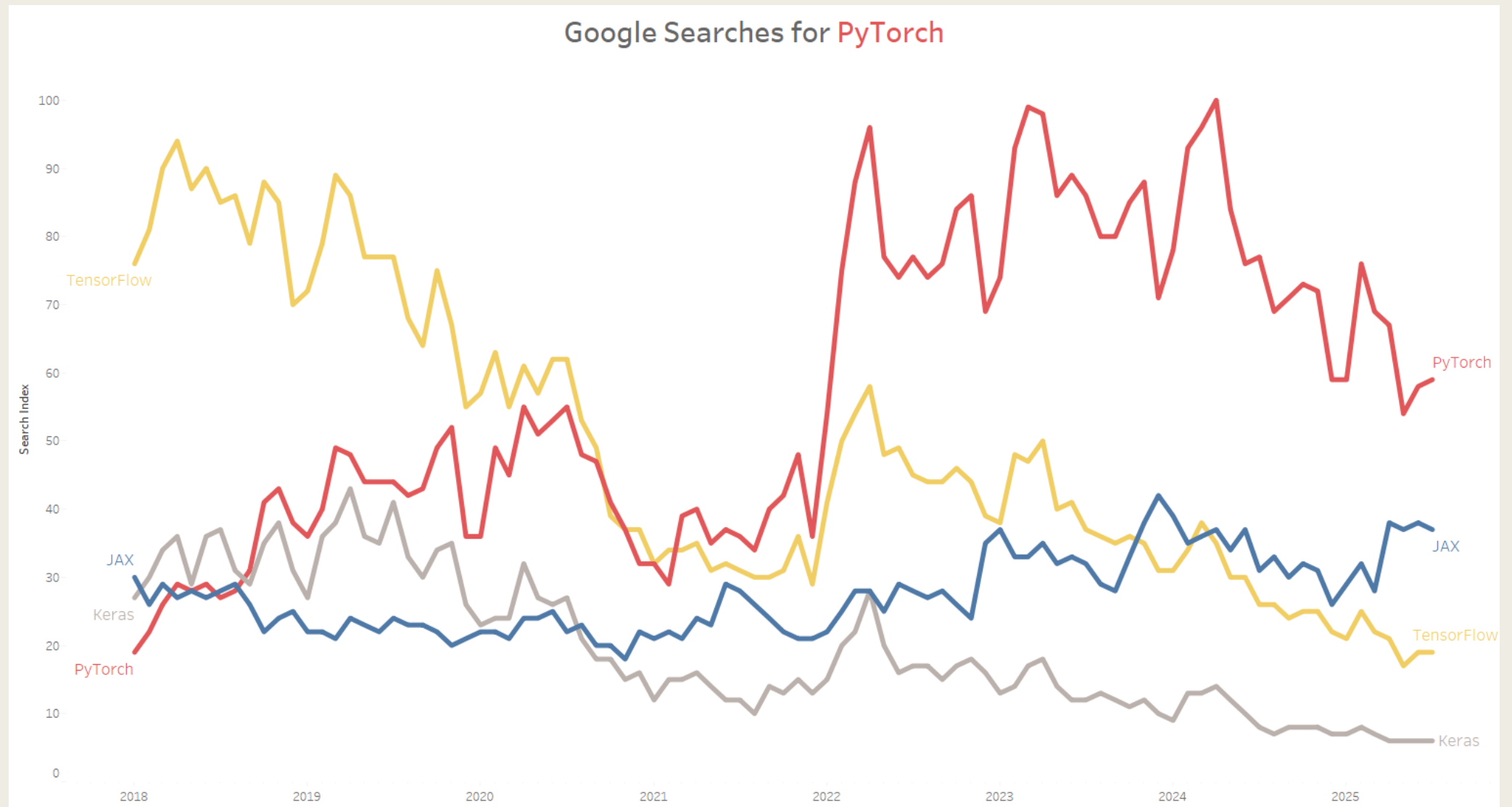
- *Vanishing/Exploding Gradients:*
 - Gradients become too small or too large.
 - Solutions: Proper initialization, batch normalization, residual connections
- *Overfitting*
 - Model memorizes training data
 - Solutions: Regularization, data augmentation, early stopping
- *Computational Requirements*
 - Training deep networks requires significant resources
 - Solutions: GPU acceleration, model compression, efficient architectures

■ Practical Challenges

- *Data Quality: Garbage in, garbage out*
- *Interpretability: Black box models*
- *Ethics: Bias, fairness, privacy concerns*

Deep Learning Frameworks

	TensorFlow	PyTorch	JAX	Keras
Created By	Google	Meta (Facebook)	Google	François Chollet (Google)
Main Strength	Full-stack ML & production	Dynamic graphs & flexibility	Autograd + high-performance	Simplicity and ease of use
Best For	Scalable deployment	Research & prototyping	Scientific computing	Beginners & fast prototyping
Execution Model	Static & Eager (TF 2.x)	Eager by default	Functional + JIT	Built on TensorFlow
Ease of Use	Medium	Medium	Low	Very High
Hardware Support	CPU, GPU, TPU	CPU, GPU	CPU, GPU, TPU	CPU, GPU, TPU (via TF)
Ecosystem	TF Lite, TFX, TF Hub	TorchVision, TorchText	NumPy-like API, Flax	Works with TensorFlow tools



Summary

- In this module we examined
 - *what deep learning is*
 - *an artificial neural network*
 - *components of neural networks*
 - *types of deep learning networks*
 - *data processing, metrics, and tuning,*
 - *challenges in deep learning, and*
 - *deep learning frameworks*